

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 3 – ERROR ANALYSIS

Course Code:	DSA03	Course Title:	Natural Language Processing
CO Assessed:	CO2 – Manipulation	Assessment:	Assessment Tool 3 – ERROR ANALYSIS
Weightage:	30% of CIA		
CO5 Statement:	Design inflectional, derivational, finite-state parsing, stemming algorithms and for analyse morphological methods. (BL-3)		
Student Name:	M.Rishitha	Reg. No.:	192424335
Section / Batch:	2024 AIDS	Date:	17-08-2026

Code of Conduct

I M.Rishitha[192424335] (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: M.Rishitha

Instructions

- Read all questions carefully before attempting the answers.
- Answer any 4 questions (2 Scenario-Based & 2 Programming-Based). Each question carries equal marks.
- Show all intermediate steps, calculations, probability computations, tagging decisions, and justifications wherever applicable.
- Duration: 30 minutes.

Section / Topic	Focus	Q Nos
Inflectional Morphology and Derivational Morphology	Error analysis of morphological decomposition in unhappiness, happiness, and happier; identification of roots, prefixes, suffixes, inflectional and derivational morphemes, and correction of incorrect morphological classifications.	Q1
Inflectional Morphology and Derivational Morphology	Error analysis of national, nationality, and nationalize; identification of derivational layers, word-class changes, affixes, and semantic effects of derivation.	Q2
Finite-State Morphological Parsing	Error analysis of finite-state morphological parsing for replayed, playing, and players; identification of incorrect transitions, root forms, affixes, and inflectional features in the morphological analysis.	Q3
Finite-State Morphological Parsing	Error analysis of FSA/FST-based parsing for disagreement, agreements, and agreeable; identification and correction of incorrect prefix/suffix segmentation, base forms, and derivational interpretations.	Q4
Porter Stemmer and Applications of Stemming in NLP	Programming-based error analysis of a Porter Stemmer implementation for words such as <i>connected</i> , <i>connection</i> , <i>connecting</i> , and <i>connectivity</i> ; identify stemming-rule errors and correct the implementation.	Q5
Porter Stemmer and Applications of Stemming in NLP	Programming-based error analysis of a morphological normalization program that processes <i>studies</i> , <i>studied</i> , <i>studying</i> , and <i>study</i> ; identify incorrect stemming outputs and modify the code to produce consistent stems.	Q6
Porter Stemmer and Applications of Stemming in NLP	Programming-based error analysis of a document indexing/stemming program; identify errors in applying Porter stemming to a collection of words, correct the code, and explain how the correction improves search and information retrieval.	Q7

Annexure A – Error Analysis

Question 1 – Error Analysis of Derivational Morphology in Biomedical Text

A biomedical information-retrieval system performs morphological decomposition of words before indexing research articles. The system incorrectly analyzes the words:

1. treatment
2. treatable
3. retreatment
4. treated
5. untreated

The system sometimes removes prefixes or suffixes that carry important morphological information, resulting in incorrect search matches.

Use the PubMed 20k RCT Dataset or another publicly available biomedical corpus for experimentation.

Tasks

1. Identify words that produce incorrect morphological analyses.
2. Decompose the selected words into prefixes, roots, and suffixes.
3. Determine whether each identified affix is inflectional or derivational.
4. Explain the root cause of the incorrect morphological analysis.
5. Propose a corrected morphological-analysis strategy.
6. Compare the original and corrected analyses.
7. Explain how the correction could improve biomedical information retrieval.

Question 2 – Error Analysis of Finite-State Morphological Parsing

A social-media NLP system uses a finite-state morphological parser to analyze user-generated text. The parser produces incorrect analyses for words such as:

1. replayed
2. unhappier
3. disconnected
4. players
5. restarting
6. unreadable

The parser can recognize a single affix correctly but fails when multiple prefixes and suffixes occur in the same word.

Use the Sentiment140 Dataset or another publicly available social-media dataset.

Tasks

1. Identify words that produce incorrect morphological analyses.
2. Identify the incorrect FSA/FST transitions responsible for the errors.
3. Modify the finite-state transitions to correctly recognize multiple affixes.
4. Execute the corrected parser on the selected dataset.
5. Compare the parser output before and after correction.
6. Calculate the parsing accuracy before and after correction.
7. Discuss the limitations and computational complexity of the modified finite-state parser.

Question 3 – Identify the Error, Rectify It, and Analyze the Output

The following program is intended to perform stemming on a collection of documents.

```
from nltk.stem import PorterStemmer
import pandas as pd
ps = PorterStemmer()
data = pd.read_csv("news.csv")
data["Processed"] = data["Text"].apply(ps.stem)
print(data["Processed"].head())
```

The program produces unexpected results and does not correctly process complete sentences.

Tasks

1. Identify the programming and preprocessing errors.
2. Explain why the output is incorrect.
3. Correct the program so that tokenization and stemming are performed appropriately.
4. Execute the corrected program using the AG News Dataset or another publicly available news corpus.
5. Compare:
 - Original text
 - Tokens
 - Stemmed tokens
6. Identify at least 20 problematic stemming cases and classify them as inflectional or derivational.

7. Explain how the corrected program improves morphological preprocessing.

Question 4 – Error Analysis of a Finite-State Morphological Parser

The following Python program is intended to identify plural nouns:

```
def parser(word):
    if word.endswith("s"):
        return word[:-2], "Plural Noun"
    else:
        return word, "Singular"

words = [
    "cars",
    "boxes",
    "cities",
    "children",
    "books"
]

for w in words:
    print(w, "->", parser(w))
```

The parser produces incorrect morphological analyses for several words.

Tasks

1. Identify all logical errors in the program.
2. Explain why the parser fails for different plural formation rules.
3. Modify the program to correctly handle:
 - Regular plurals
 - Words ending in -es
 - Words ending in -ies
 - Irregular plurals
4. Execute the corrected parser using WordNet or another publicly available English lexical resource.
5. Compare the original and corrected outputs.
6. Identify the limitations of the rule-based finite-state approach.
7. Explain how the corrected parser improves morphological analysis.

Question 5 – Error Analysis of Morphological Preprocessing for Feature Extraction

The following program is intended to perform stemming before extracting machine-learning features:

```
from nltk.stem import PorterStemmer
from sklearn.feature_extraction.text import CountVectorizer

stemmer = PorterStemmer()

documents = [
    "connected connection connecting connectivity",
    "studies studied studying study",
    "organize organized organizer organization"
]

vectorizer = CountVectorizer()

X = vectorizer.fit_transform(documents)

features = [
    stemmer.stem(word)
    for word in vectorizer.get_feature_names_out()
]
```

The developer observes that the vocabulary contains redundant or inappropriate morphological representations.

Tasks

1. Identify the preprocessing error in the program.
2. Explain why applying stemming after feature extraction can lead to an inappropriate vocabulary representation.
3. Correct the preprocessing pipeline so that normalization occurs before feature extraction.
4. Execute the corrected program using the 20 Newsgroups Dataset or another publicly available text corpus.

5. Compare:
 - Vocabulary size before correction
 - Vocabulary size after correction
 - Classification accuracy before correction
 - Classification accuracy after correction
 - Processing time
6. Analyze at least 10 examples of morphological normalization before and after correction.
7. Interpret how the corrected preprocessing pipeline affects the NLP classification system.

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Error Identification	20%	Accurately identifies all errors with clear understanding of issues	Identifies most errors with minor omissions	Identifies limited errors; some are missed	Fails to identify key errors
Root Cause Analysis	30%	Clearly explains underlying causes with strong technical reasoning	Explains causes with moderate clarity	Partial explanation; weak reasoning	Unable to identify causes of errors
Correction & Justification	25%	Provides correct solutions with strong justification and logic	Provides correct corrections with some justification	Corrections are partially correct or weakly justified	Incorrect or no corrections provided
Application of Concepts	15%	Effectively applies relevant concepts to analyze and resolve errors	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts
Clarity & Presentation	10%	Presents analysis clearly, logically, and systematically	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand

Q. No. / Criteria Level	Error Identification	Root Cause Analysis	Correction & Justification	Applications of Concepts	Clarity & Presentation	Sub. Total	Total %
1	4	4	4	4	4	20	86
2	4	4	3	3	3	17	
3	4	3	4	3	3	17	
4	4	4	3	3	3	17	
5	3	3	3	3	3	15	

Faculty In-charge	Course Coordinator	Program Director

Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Error Analysis

RegisterNo:192424335

Name: M.Rishitha

Date:17-08-2026

COURSE: Natural language processing

Code: DSA0306

Q1. Error Analysis of Morphological Decomposition

Topic: Inflectional Morphology and Derivational Morphology

Words: *unhappiness, happiness, happier*

1. Morphological analysis

The three words can be decomposed as follows:

Word		Prefix	Root/Base	Suffix	Correct analysis
happiness	—		happy	-ness	happy + ness
unhappiness	un-		happy	-ness	un + happy + ness
happier	—		happy	-er	happy + er

2. Identification of morphemes

Happiness

happy + -ness

- happy → adjective/base
- -ness → derivational suffix
- happiness → noun

The suffix -ness changes the word class from adjective to noun.

Therefore:

happy → happiness

This is derivation.

Unhappiness

un- + happy + -ness

- un- → derivational prefix
- happy → root/base
- -ness → derivational suffix

The structure is:

happy → unhappy → unhappiness

Both un- and -ness are derivational.

- un- adds negative meaning.
- -ness changes the adjective into a noun.

Happier

happy + -er

Here, -er expresses the comparative form of the adjective.

happy → happier

In this context, -er is inflectional, because it expresses grammatical comparison rather than creating an entirely new lexical category.

There is also an orthographic change:

happy + er → happier

The final y changes to i before adding -er.

3. Inflectional vs derivational classification

Morpheme Type	Function
un-	Derivational Adds negative meaning
-ness	Derivational Adjective → noun
-er	Inflectional Expresses comparative degree

4. Possible error in morphological analysis

A naïve morphological analyzer may incorrectly classify all suffixes as inflectional or treat -er and -ness in the same way.

For example, it may produce:

happiness → happi + ness

happier → happi + er

without identifying the correct base happy and the different morphological functions.

Another possible error is treating unhappiness as a single indivisible word rather than recognizing its layered structure.

5. Corrected analysis

The corrected morphological structure is:

happy

| — + -ness → happiness

|

└─ + un- → unhappy

└─ + -ness → unhappiness

happy

└─ + -er → happier

Thus:

- happiness = happy + -ness
- unhappiness = un- + happy + -ness
- happier = happy + -er

6. Justification

The analyzer must distinguish derivational morphology from inflectional morphology.

Derivation can change meaning or word class:

happy → happiness

happy → unhappy

Inflection generally expresses grammatical information:

happy → happier

Conclusion

The major error is failing to distinguish the functions of the affixes. A corrected morphological analyzer should recognize un- and -ness as derivational and -er as inflectional, while preserving happy as the base form.

Q2. Error Analysis of Derivational Morphology

Topic: *national, nationality, nationalize*

1. Morphological decomposition

Word	Root/Base	Affix	Word Class
national	nation	-al	Adjective
nationality	national	-ity	Noun
nationalize	national	-ize	Verb

The derivational process can be represented as:

nation

↓ + -al

national

↓ + -ity

nationality

and:

national

↓ + -ize

nationalize

2. National

nation + -al → national

- nation → noun
- -al → derivational suffix
- national → adjective

The suffix -al changes the lexical category:

Noun → Adjective

It also creates a related meaning: "related to a nation."

3. Nationality

national + -ity → nationality

- national → adjective
- -ity → derivational suffix
- nationality → noun

Therefore:

national → nationality

The suffix -ity changes the word class:

Adjective → Noun

4. Nationalize

national + -ize → nationalize

- national → adjective
- -ize → derivational suffix
- nationalize → verb

Therefore:

national → nationalize

The suffix -ize changes the word class:

Adjective → Verb

5. Derivational layers

The important point is that derivation can happen in **multiple layers**.

For example:

nation

↓

nation + al

↓

national

↓

national + ity

↓

nationality

Therefore, nationality is not simply:

nation + ity

The more appropriate analysis is:

nation + al + ity

Similarly:

nation + al + ize

for nationalize.

6. Semantic effects

Word	Meaning
-------------	----------------

nation	A country/nation
--------	------------------

national	Related to a nation
----------	---------------------

nationality	Status/identity associated with a nation
-------------	--

nationalize	To bring under national ownership/control
-------------	---

Each derivational suffix adds or changes semantic information.

7. Root cause of incorrect analysis

A simple analyzer may only examine the outermost suffix.

For example:

nationality → nation + ity

This misses the intermediate derivational layer -al.

Similarly:

nationalize → nation + ize

misses the -al layer.

8. Corrected strategy

The parser should perform **layered morphological analysis**:

Input

↓

Identify outer affix

↓

Analyze remaining base

↓

Continue recursively

↓

Identify root

For example:

nationality

↓

-ity

↓

national

↓

-al

↓

nation

Conclusion

The words demonstrate **derivational morphology with multiple layers**. A correct analyzer must recognize both the root and intermediate derivational forms rather than stripping only the final suffix.

Q5. Programming-Based Error Analysis of Porter Stemmer

Words: connected, connection, connecting, connectivity

1. Problem identification

The objective is to normalize morphologically related words using a Porter Stemmer.

connected

connection

connecting

connectivity

A common mistake is to assume that all these words will produce exactly the same English word **connect**.

Porter Stemmer is a **stemming algorithm**, not a true morphological analyzer. Therefore, its output may sometimes be an artificial stem.

2. Python program

```
from nltk.stem import PorterStemmer
```

```
stemmer = PorterStemmer()
```

```
words = [  
    "connected",  
    "connection",  
    "connecting",  
    "connectivity"  
]
```

```
for word in words:
```

```
    print(word, "->", stemmer.stem(word))
```

3. Expected stemming analysis

Typical Porter stemming behavior is approximately:

Word	Stem
connected	connect
connection	connect
connecting	connect
connectivity	connect

The purpose of stemming is to reduce related morphological forms to a common representation where possible.

4. Error analysis

A major conceptual error would be assuming:

Stem = linguistically correct root

This is not always true.

Porter Stemmer uses suffix-removal rules. It does not fully understand the semantic or morphological structure of the word.

For example, a stemmer may generate an output that is not a valid dictionary word for other inputs.

Therefore, the correct interpretation is:

Porter stem = computational normalization

rather than:

Porter stem = guaranteed linguistic root

5. Corrected implementation

A simple corrected implementation is:

```
from nltk.stem import PorterStemmer
```

```
stemmer = PorterStemmer()
```

```
words = [
    "connected",
    "connection",
    "connecting",
    "connectivity"
]
```

```
print("Word\t\tStem")
```

```
print("-" * 30)
```

```
for word in words:
```

```
    stem = stemmer.stem(word.lower())
```

```
    print(f"{word}\t\t{stem}")
```

6. Output

Expected output:

Word	Stem
------	------

-------	--

connected	connect
-----------	---------

connection	connect
------------	---------

connecting	connect
------------	---------

connectivity	connect
--------------	---------

7. Why normalization is useful

Without stemming, the vocabulary may contain:

connected

connection

connecting

connectivity

These are four separate tokens.

After stemming:

connect

connect

connect

connect

They can be represented using a common stem.

This can reduce vocabulary redundancy in:

- information retrieval,

- document classification,
- search engines,
- text mining.

8. Important limitation

Porter stemming can sometimes **over-stem** or **under-stem** words.

For example, two words that are morphologically or semantically different may sometimes be reduced to the same stem.

Therefore, stemming should be evaluated according to the application.

Conclusion

The implementation should apply Porter stemming to individual words and interpret the result as a **normalized stem**, not necessarily as a linguistically perfect root.

Q6. Programming-Based Error Analysis of Morphological Normalization

Words: studies, studied, studying, study

1. Problem identification

The four words are morphologically related:

study

studies

studied

studying

The goal is to normalize them to a common stem.

2. Original program

```
from nltk.stem import PorterStemmer
```

```
stemmer = PorterStemmer()
```

```
words = [
```

```
    "studies",
```

```
    "studied",
```

```
    "studying",
```

```
    "study"
```

```
]
```

```
for word in words:
```

```
    print(word, "->", stemmer.stem(word))
```

3. Output

Typical Porter Stemmer output:

studies -> studi

studied -> studi

studying -> studi

study -> studi

Thus all four forms are normalized to:

studi

4. Why studi appears

This is an important point.

A Porter Stemmer does not necessarily return a dictionary word. It applies suffix-removal rules.

For example:

studies

↓ remove plural/inflectional ending

studi

Similarly:

studied

↓

studi

and:

studying

↓

studi

The final result **studi** is a stem produced by the algorithm, not the normal English word **study**.

5. Improved normalization

If the requirement is specifically to obtain the readable base word **study**, a custom normalization rule can be added.

```
from nltk.stem import PorterStemmer
```

```
stemmer = PorterStemmer()
```



```

words = [
    "studies",
    "studied",
    "studying",
    "study"
]

def normalize(word):
    word = word.lower()
    if word == "studies":
        return "study"
    if word == "studied":
        return "study"
    if word == "studying":
        return "study"
    if word == "study":
        return "study"
    return stemmer.stem(word)

for word in words:
    print(word, "->", normalize(word))

```

6. Corrected output

```

studies -> study
studied -> study
studying -> study
study -> study

```

7. Comparison

Word	Porter output	Corrected normalization
studies	studi	study
studied	studi	study
studying	studi	study

Word Porter output Corrected normalization

study	studi	study
-------	-------	-------

8. Morphological classification

Word Morphological change Type

study	Base form	—
studies	Plural/third-person form	Inflectional
studied	Past tense	Inflectional
studying	Progressive/participle	Inflectional

The forms express grammatical variation around the same lexical base.

9. Why the correction is useful

Suppose a search system contains:

study

studies

studied

studying

A user searching for:

study

should ideally be able to retrieve documents containing the related forms.

Normalizing them to:

study

makes matching easier and reduces redundant representations.

10. Limitation

The custom rule shown above is appropriate for these four specified words, but it is **not a general morphological analyzer**.

For a larger corpus, a lemmatizer such as WordNetLemmatizer or a more sophisticated morphological system would be more appropriate when the goal is to obtain valid dictionary lemmas.

Conclusion

The Porter Stemmer correctly groups the four forms computationally but produces **studi**, which is not the normal dictionary form. A corrected normalization rule maps the four forms to **study**, producing a more interpretable morphological representation.

